

FICHE DE TD 3/3 • DIFFICILE

Informatique • 3ème

Utilisation des concepts de base du développement logiciel

Informatique – classe de 3ème

Cette fiche d'évaluation synthétique mobilise l'algorithmique, la modularité, l'analyse de complexité et la conception de jeux de tests. Toute réponse doit être justifiée rigoureusement.

EXERCICE 1 Analyse et correction d'un algorithme de tri hybride

On considère l'algorithme suivant, censé trier un tableau T de n entiers par ordre croissant en combinant une passe de tri à bulles et une recherche dichotomique pour l'insertion.

```
Algorithme TriHybride
Données :  $T[0..n-1]$  : Tableau d'entiers,  $n$  : Entier ( $n > 1$ )
Résultat :  $T$  trié sur place
Variables :  $i, j, k, cle, gauche, droite, milieu$  : Entiers
Début
  Pour  $i$  de 1 à  $n-1$  faire
     $cle \leftarrow T[i]$ 
     $gauche \leftarrow 0$ 
     $droite \leftarrow i - 1$ 
    // Recherche dichotomique de la position d'insertion
    TantQue  $gauche \leq droite$  faire
       $milieu \leftarrow (gauche + droite) \text{ div } 2$ 
      Si  $T[milieu] > cle$  Alors
         $droite \leftarrow milieu - 1$ 
      Sinon
         $gauche \leftarrow milieu + 1$ 
      FinSi
    FinTantQue
    // Décalage des éléments
    Pour  $j$  de  $i-1$  à  $gauche$  par pas de  $-1$  faire
       $T[j+1] \leftarrow T[j]$ 
    FinPour
     $T[gauche] \leftarrow cle$ 
  FinPour
Fin
```

1. **Traçabilité** : Effectuer la trace complète de l'algorithme pour le tableau initial $T = [5, 2, 8, 1]$ ($n = 4$). Présenter l'état du tableau après chaque itération de la boucle principale (Pour i).
2. **Correction** : L'algorithme ci-dessus contient une erreur logique dans la gestion des bornes lors de la recherche dichotomique pour le cas où cle est plus grand que tous

les éléments de la partie triée. Identifier l'erreur, l'expliquer et proposer la correction exacte de la ligne concernée.

3. Invariant de boucle : Énoncer et démontrer par récurrence l'invariant de la boucle principale (Pour i) qui garantit la correction partielle du tri (hors bug identifié).
4. Complexité : Déterminer la complexité temporelle dans le pire cas en notation $O(\cdot)$. Justifier si l'utilisation de la dichotomie améliore la complexité asymptotique globale par rapport à un tri par insertion classique.

EXERCICE 2 Conception modulaire : Gestion d'un stock de médicaments

Une pharmacie gère son stock via une structure de données `Medicament` (chaîne nom, entier quantite, réel prixUnitaire, date peremption). Le stock est un tableau dynamique `Stock[0..N-1]` trié par ordre alphabétique des noms.

1. Spécification formelle : Rédiger la spécification complète (Pré-condition, Post-condition) d'une fonction `RechercherDichotomique(nom : Chaîne) → Entier` renvoyant l'indice du médicament ou `-1` si absent. Préciser l'invariant de la boucle de recherche.
2. Algorithme modulaire : Écrire en pseudo-code normalisé une procédure `Vendre(nom : Chaîne, qte : Entier)` qui :
 - utilise `RechercherDichotomique`;
 - vérifie la disponibilité ($\text{quantité} \geq \text{qte}$) et la non-péremption ($\text{date} > \text{date du jour}$);
 - met à jour le stock et affiche le montant total;
 - gère les cas d'erreur (introuvable, stock insuffisant, périmé) par des messages explicites sans planter.
3. Jeu de tests de non-régression : Concevoir un tableau de 6 cas de tests significatifs (valeurs limites, partitions d'équivalence, cas nominaux) pour valider `Vendre`. Pour chaque cas : décrire l'état initial du stock, les paramètres d'appel, le résultat attendu (retour, affichage, état final du stock).
4. Évolution : Le pharmacien souhaite désormais regrouper les médicaments par `familleThérapeutique`. Proposer une modification de la structure de données (typage) et expliquer l'impact algorithmique sur la recherche dichotomique. Justifier si le tri alphabétique global reste pertinent.

EXERCICE 3 Modélisation et analyse critique : Système de réservation de places

Un cinéma dispose d'une salle de L lignes et C colonnes. L'occupation est modélisée

par une matrice booléenne `Salle[1..L][1..C]` (Vrai = occupé). On souhaite implémenter une fonction `MeilleurBloc(k : Entier) → (Entier, Entier)` renvoyant les coordonnées (ligne, colonne de départ) du premier bloc de k places contiguës horizontalement disponibles sur une même ligne, en privilégiant les lignes les plus basses (écran), puis la gauche.

1. **Algorithme optimal** : Proposer un algorithme en pseudo-code pour `MeilleurBloc`. La complexité temporelle doit être en $O(L \times C)$. Interdiction de tester toutes les positions de départ naïvement ($O(L \times C \times k)$). Expliquer la stratégie utilisée (fenêtre glissante / comptage).
2. **Preuve de correction** : Définir l'invariant de la boucle interne (parcours d'une ligne) qui garantit la détection du premier bloc de taille k . Démontrer l'initialisation, la conservation et la terminaison.
3. **Analyse critique d'une proposition** : Un élève propose le code suivant pour le parcours d'une ligne :

```
compteur ← 0
Pour j de 1 à C faire
    Si Non Salle[i][j] Alors compteur ← compteur + 1
    Sinon compteur ← 0
    Si compteur = k Alors Renvoyer (i, j - k + 1)
FinPour
```

Ce code est-il correct pour la spécification « premier bloc en partant de la gauche » ? Si non, donner un contre-exemple précis (valeurs de C , k , état de la ligne) et corriger l'algorithme.

4. **Gestion de la fragmentation** : Après de nombreuses réservations/annulations, la salle est fragmentée. Proposer une structure de données complémentaire (à maintenir à jour) permettant de répondre à `MeilleurBloc(k)` en $O(L)$ ou $O(\log L)$ indépendamment de C . Décrire la mise à jour lors d'une réservation/annulation.